

Companion-AI Core Loop: Memory & Evaluation

Tech Generalist candidate

1. The problem

AI companion products — like ira and Replika — all promise the same two things: a persona that feels consistent over time, and a companion that "remembers you." In practice, most of them visibly fail at both after enough turns: the persona contradicts an earlier stated opinion, forgets something the user told it last week, or resets to a generic assistant voice under pressure.

We want you to build a small, working system that actually solves this — not a chatbot with a system prompt, but a real memory architecture with retrieval, update, and contradiction-handling logic — and then prove it works with an evaluation harness, since "it felt consistent when I tried it" isn't evidence.

This task is scoped to exactly that: the memory and personality-consistency architecture is the primary focus (see §2). Building an evaluation harness for it is a secondary, optional stretch goal if time allows (see §3). Everything else — UI, auth, voice, images, infrastructure scale — is explicitly out of scope (see §4). We want to see the core loop and how you think, not a polished product.

2. The core loop (primary deliverable)

Build a command-line or minimal script-based chat loop (no UI required) with a defined companion persona, backed by an LLM of your choice. It should demonstrably do the following:

- **Persist across sessions.** Facts and context should survive a process restart — this is not just "conversation history in the context window."
- **Extract and store memory.** Define what counts as a memory-worthy fact, how it's extracted from conversation, and where/how it's stored (structured facts, embeddings, hybrid — your choice, with reasoning).
- **Retrieve relevantly.** Recall the right memories at the right time — not dump everything into context, and not miss something clearly relevant.
- **Update and decay.** Handle contradiction: e.g. the user says something that supersedes an earlier fact ("I broke up with my ex" after earlier mentioning the relationship). Old fact should be updated or retired, not just added alongside.
- **Stay in character.** The persona's own stated traits, opinions, and backstory should not contradict themselves over 50+ turns, and shouldn't flatten into a generic-assistant tone under topic pressure.

***A note on content:** the persona should be a warm, companion-type character — that's the point of the exercise — but the test conversations do not need to be romantic or explicit to exercise memory and consistency. Keep test transcripts focused on everyday personal disclosure (relationships, work stress, plans, opinions) rather than intimate content; that keeps the focus on the systems problem, which is what we're evaluating.*

3. The evaluation harness (optional — only if time allows)

Building the core loop well within 2 days will likely take most of your time, and that's fine — it's the priority. If you have time left over, a lightweight evaluation harness is a nice-to-have, not a requirement. There's no ground-truth dataset for "is this a good companion response," so any evidence you can produce is a bonus. If you get to it:

- **A test conversation set** (can be synthetic / self-generated) that deliberately exercises memory recall, contradiction handling, and long-range consistency — e.g. state a fact early, revisit it 40 turns later, contradict it, check what happens.
- **Automated detection** of memory failures (forgetting, wrong recall) and personality drift (contradicted traits, tone flattening) — an LLM-as-judge rubric is a reasonable approach, but explain the rubric and its limitations.
- **Results with numbers.** Pass/fail rates, example failures, and your own read on where the system is weakest.
- **Optional but valued:** an "oracle" baseline — e.g. a strong reasoning model given the full memory store and asked what the ideal recall/response would be — to compare the system against.

Don't sacrifice time on the core loop to get to this — a strong core loop with no eval harness is a better submission than a weak core loop with one.

4. Explicitly out of scope

- UI/UX polish — a terminal loop or a single unstyled script is completely fine
- Authentication, billing, multi-user support
- Voice, image, or video generation
- Production-scale infra, load handling, or latency optimization

5. Deliverables

- Source code for the working prototype, runnable from a README
- A short README covering: architecture decisions and why, what was tried and abandoned, known limitations
- The eval harness, test conversation set, and results, if you got to it (numbers + example failures)
- A 15–20 minute walkthrough (recorded video or live in the follow-up interview) demoing the core loop, plus eval results if you have them

6. Logistics

- **Time estimate:** roughly 18 hours of focused work. We're not expecting a finished product — partial progress with clear reasoning beats a rushed complete system.
- **Model/provider choice** is up to you; we can provide API credits on request.
- **Submit** as a git repo (private, shared with us) or zip — whichever is faster for you.